Mathematical framing

In this section, I study the model as a function $\hat{p}(x)$ mapping inputs to probabilities. I interpret each computation as evaluating properties of this function such as gradients, perturbations, or expectations.

XGBoost XAI study notebook

This notebook is a model-specific XAI study for the trained `view_e_full` XGBoost pipeline reconstructed from `churn_analysis_helpers.py`.

It is intentionally organised around the model's function class rather than only around predictive replay.

Study goals

We treat the fitted model as

$$f_{\mathrm{xgb}}(x)=\sum_{m=1}^{M} \eta T_m(x),\ p_{\mathrm{xgb}}(x)=\sigma\left(f_{\mathrm{xgb}}(x)\right).$$

Because each tree induces axis-aligned splits, the fitted logit surface is piecewise constant inside leaf regions. The central questions are therefore:

1. Which engineered features dominate global risk allocation?
2. Which interactions create the sharpest regime changes?
3. Where does the model become overconfident or brittle?
4. What do its conditional decision surfaces actually look like?

Required files in `./`

- `train.csv`
- `test.csv`
- `WA_Fn-UseC_-Telco-Customer-Churn.csv`
- `xgb_view_e_full_final_model.json`
- `xgb_view_e_full_feature_columns.json`
- `xgb_view_e_full_training_metadata.json`
- `churn_analysis_helpers.py`

```
from pathlib import Path
import json
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import shap
import xgboost as xgb

from sklearn.metrics import roc_auc_score, brier_score_loss, log_loss
from sklearn.inspection import PartialDependenceDisplay
from sklearn.calibration import calibration_curve

from churn_analysis_helpers import (
    CFG,
    seed_everything,
    setup_plotting,
    ensure_output_dir,
    savefig,
    build_feature_frames,
    build_xgb_runtime_frames,
    load_xgb_artifacts,
    align_xgb_features,
    pred_summary,
    make_decile_frame,
    cumulative_capture,
    compute_xgb_shap_values,
)

seed_everything(67)
setup_plotting()

cfg = CFG()
ensure_output_dir(cfg)
```

Mathematical framing

In this section, I study the model as a function $\hat{p}(x)$ mapping inputs to probabilities. I interpret each computation as evaluating properties of this function such as gradients, perturbations, or expectations.

1. Reconstruct the exact XGBoost runtime feature space

This section does not create a new feature pipeline. It uses the helper exactly as written, rebuilds the runtime frame, loads the saved model artefacts, aligns the runtime columns to the saved feature column order, and obtains fitted probabilities on train and test.

```
bundle = build_feature_frames(cfg)
runtime = build_xgb_runtime_frames(bundle)

xgb_model, feature_cols_saved, xgb_metadata = load_xgb_artifacts(cfg)

X_train_aligned, extra_train_cols =
```

```
align_xgb_features(runtime["X_train_final"], feature_cols_saved)
X_test_aligned, extra_test_cols =
align_xgb_features(runtime["X_test_final"], feature_cols_saved)

xgb_train_pred = xgb_model.predict_proba(X_train_aligned)[:, 1]
xgb_test_pred = xgb_model.predict_proba(X_test_aligned)[:, 1]

n_comp = len(bundle["y"])
comp_mask = np.arange(len(X_train_aligned)) < n_comp

xgb_comp_train_pred = xgb_train_pred[comp_mask]
y = bundle["y"]

print("Saved feature columns:", len(feature_cols_saved))
print("Runtime aligned train shape:", X_train_aligned.shape)
print("Runtime aligned test shape :", X_test_aligned.shape)
print("Competition-train AUC:", f"{roc_auc_score(y,
xgb_comp_train_pred):.6f}")
print("Competition-train log loss:", f"{log_loss(y,
xgb_comp_train_pred):.6f}")
print("Competition-train Brier:", f"{brier_score_loss(y,
xgb_comp_train_pred):.6f}")
print("Extra runtime cols ignored:", len(extra_train_cols))
display(pd.Series(pred_summary(xgb_comp_train_pred,
"xgb_train")).to_frame("value"))
```

```
Saved feature columns: 161
Runtime aligned train shape: (601237, 161)
Runtime aligned test shape : (254655, 161)
Competition-train AUC: 0.933437
Competition-train log loss: 0.268055
Competition-train Brier: 0.085993
Extra runtime cols ignored: 15
```

```
                      value
xgb_train_min      0.000003
xgb_train_p01      0.000015
xgb_train_p05      0.000050
xgb_train_median   0.053924
xgb_train_mean     0.225706
xgb_train_p95      0.840071
xgb_train_p99      0.958884
xgb_train_max      0.999883
```

```
analysis_train = pd.DataFrame({
    cfg.id_col: bundle["train"][cfg.id_col].values,
    "y_true": y,
    "xgb_pred": xgb_comp_train_pred,
})
```

```
analysis_test = pd.DataFrame({
    cfg.id_col: bundle["test"][cfg.id_col].values,
    "xgb_pred": xgb_test_pred,
})

analysis_train["pred_rank_pct"] =
analysis_train["xgb_pred"].rank(pct=True, method="average")
analysis_train["error"] = analysis_train["y_true"] -
analysis_train["xgb_pred"]
analysis_train["abs_error"] = analysis_train["error"].abs()
analysis_train["hard_label"] = (analysis_train["xgb_pred"] >=
0.5).astype(int)
analysis_train["case_type"] = np.select(
    [
        (analysis_train["y_true"] == 1) &
(analysis_train["hard_label"] == 1),
        (analysis_train["y_true"] == 0) &
(analysis_train["hard_label"] == 0),
        (analysis_train["y_true"] == 0) &
(analysis_train["hard_label"] == 1),
        (analysis_train["y_true"] == 1) &
(analysis_train["hard_label"] == 0),
    ],
    ["TP", "TN", "FP", "FN"],
    default="other"
)

analysis_train["risk_band"] = pd.qcut(
    analysis_train["xgb_pred"],
    q=5,
    labels=["very_low", "low", "mid", "high", "very_high"],
    duplicates="drop",
)

analysis_train.head()
```

```
   id  y_true  xgb_pred  pred_rank_pct     error  abs_error
hard_label  \
0   0       0  0.000078       0.078251 -0.000078   0.000078
0
1   1       0  0.001616       0.197323 -0.001616   0.001616
0
2   2       0  0.324847       0.705830 -0.324847   0.324847
0
3   3       1  0.683729       0.878599  0.316271   0.316271
1
4   4       1  0.775512       0.921608  0.224488   0.224488
1

  case_type  risk_band
```

```
0         TN   very_low
1         TN   very_low
2         TN       high
3         TP  very_high
4         TP  very_high
```

Mathematical framing

In this section, I study the model as a function $\hat{p}(x)$ mapping inputs to probabilities. I interpret each computation as evaluating properties of this function such as gradients, perturbations, or expectations.

2. Predictive and calibration diagnostics

This is the behavioural layer of the study.

We inspect:

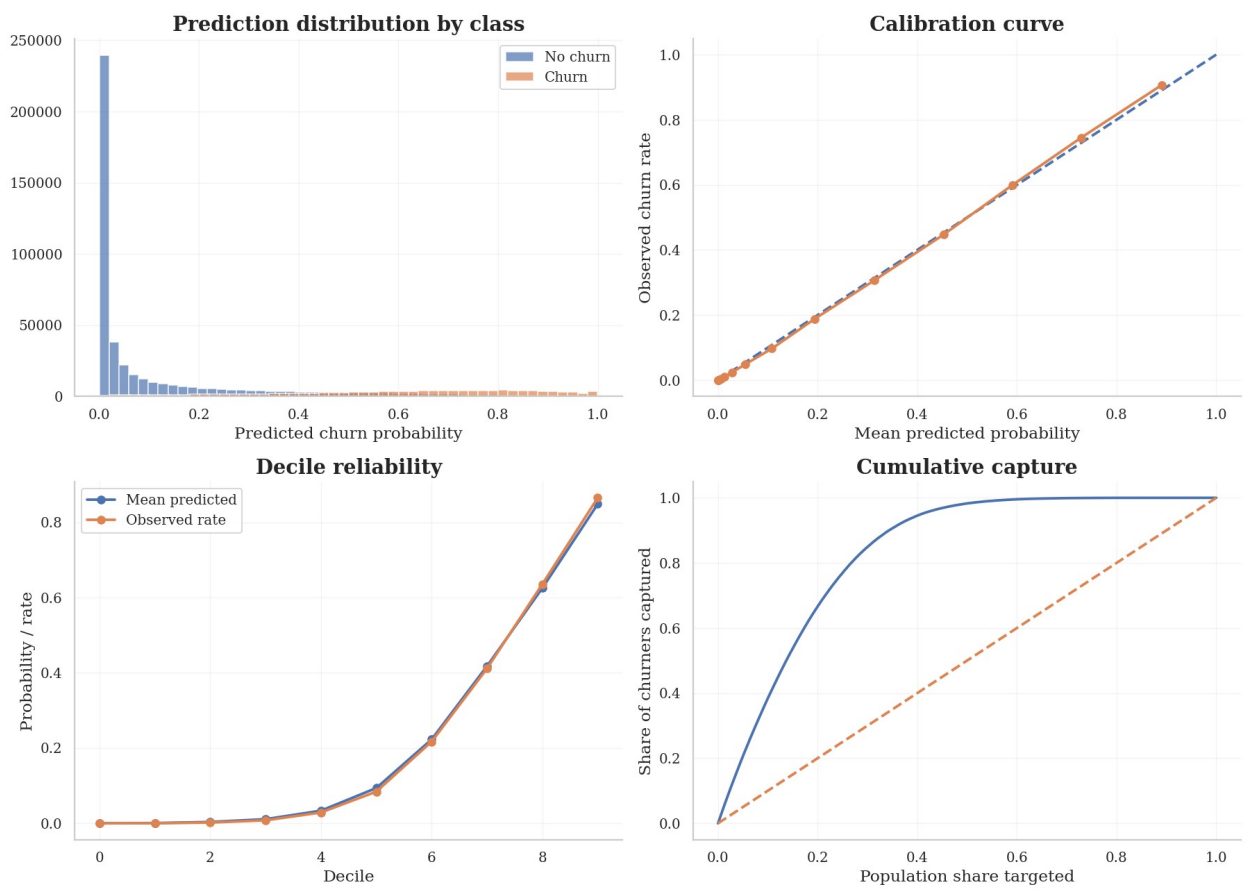
- score distribution,
- empirical reliability,
- decile-level calibration,
- cumulative capture,
- error concentration in the tails.

If the model is exploiting sharp tree partitions well, high-risk bins should produce strong capture and the top tail should contain a large share of positive churn cases.

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

axes[0, 0].hist(analysis_train.loc[analysis_train["y_true"] == 0,
"xgb_pred"], bins=50, alpha=0.7, label="No churn")
axes[0, 0].hist(analysis_train.loc[analysis_train["y_true"] == 1,
"xgb_pred"], bins=50, alpha=0.7, label="Churn")
axes[0, 0].set_title("Prediction distribution by class")
axes[0, 0].set_xlabel("Predicted churn probability")
axes[0, 0].legend()

prob_true, prob_pred = calibration_curve(analysis_train["y_true"],
analysis_train["xgb_pred"], n_bins=15, strategy="quantile")
axes[0, 1].plot([0, 1], [0, 1], linestyle="--")
axes[0, 1].plot(prob_pred, prob_true, marker="o")
axes[0, 1].set_title("Calibration curve")
axes[0, 1].set_xlabel("Mean predicted probability")
axes[0, 1].set_ylabel("Observed churn rate")

dec = make_decile_frame(analysis_train["y_true"],
analysis_train["xgb_pred"], n_bins=10)
axes[1, 0].plot(dec["mean_pred"], marker="o", label="Mean predicted")
axes[1, 0].plot(dec["churn_rate"], marker="o", label="Observed rate")
```

```
axes[1, 0].set_title("Decile reliability")
axes[1, 0].set_xlabel("Decile")
axes[1, 0].set_ylabel("Probability / rate")
axes[1, 0].legend()

cap = cumulative_capture(analysis_train["y_true"],
analysis_train["xgb_pred"])
axes[1, 1].plot(cap["population_share"], cap["capture_rate"])
axes[1, 1].plot([0, 1], [0, 1], linestyle="--")
axes[1, 1].set_title("Cumulative capture")
axes[1, 1].set_xlabel("Population share targeted")
axes[1, 1].set_ylabel("Share of churners captured")

plt.tight_layout()
savefig(cfg, "xgb_01_core_diagnostics.png")
plt.show()

display(dec)
```

```
saved: analysis_outputs\xgb_01_core_diagnostics.png
```



```
   bin  mean_pred  churn_rate      n  pred_min  pred_max
0    0   0.000052    0.000000  59420  0.000003  0.000110
1    1   0.000682    0.000236  59419  0.000110  0.001696
2    2   0.003563    0.002003  59419  0.001696  0.005920
3    3   0.011040    0.007927  59420  0.005920  0.018740
4    4   0.033567    0.028728  59419  0.018741  0.053924
5    5   0.093750    0.084165  59419  0.053925  0.145461
6    6   0.223051    0.217115  59420  0.145461  0.313004
7    7   0.417723    0.411215  59419  0.313006  0.523134
8    8   0.625240    0.635672  59419  0.523141  0.727878
9    9   0.848388    0.865012  59420  0.727878  0.999883
```

```
tail_summary = pd.DataFrame({
    "segment": ["Top 1%", "Top 5%", "Top 10%", "Bottom 10%"],
    "observed_churn_rate": [
        analysis_train.loc[analysis_train["pred_rank_pct"] >= 0.99,
"y_true"].mean(),
        analysis_train.loc[analysis_train["pred_rank_pct"] >= 0.95,
"y_true"].mean(),
        analysis_train.loc[analysis_train["pred_rank_pct"] >= 0.90,
"y_true"].mean(),
        analysis_train.loc[analysis_train["pred_rank_pct"] <= 0.10,
"y_true"].mean(),
    ],
    "n": [
        int((analysis_train["pred_rank_pct"] >= 0.99).sum()),
        int((analysis_train["pred_rank_pct"] >= 0.95).sum()),
        int((analysis_train["pred_rank_pct"] >= 0.90).sum()),
        int((analysis_train["pred_rank_pct"] <= 0.10).sum()),
    ],
})
display(tail_summary)
```

```
      segment  observed_churn_rate      n
0      Top 1%             0.990071   5942
1      Top 5%             0.928879  29710
2     Top 10%             0.865012  59420
3  Bottom 10%             0.000000  59419
```

Mathematical framing

In this section, I study the model as a function $\hat{p}(x)$ mapping inputs to probabilities. I interpret each computation as evaluating properties of this function such as gradients, perturbations, or expectations.

3. Global SHAP structure

For the tree model, SHAP is the primary global explanation tool. We compute SHAP values on a competition-train sample and study:

- mean absolute contribution magnitude,
- signed contribution direction,
- summary structure across the sample.

For a tree ensemble, SHAP decomposes the fitted logit additively:

$$f_{\mathrm{xgb}}(x) = \phi_0 + \sum_{j=1}^{p} \phi_j(x).$$

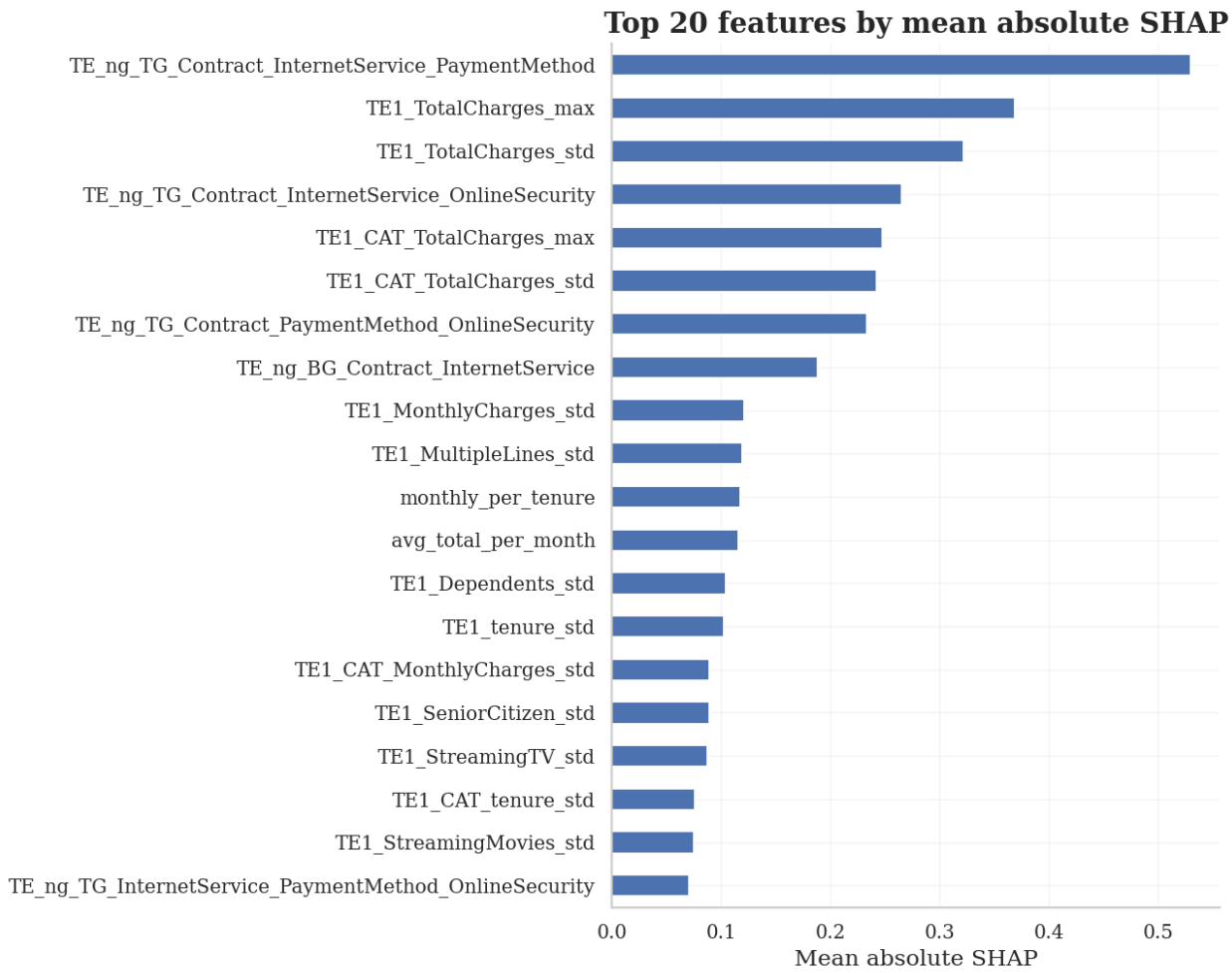
Large $|\phi_j(x)|$ means feature j is strongly moving the logit away from baseline for that case.

```
shap_idx = np.arange(n_comp)
if len(shap_idx) > cfg.shap_sample_n:
    rng = np.random.default_rng(cfg.seed)
    shap_idx = rng.choice(shap_idx, size=cfg.shap_sample_n,
replace=False)

X_shap = X_train_aligned.iloc[shap_idx].copy()
y_shap = y[shap_idx]

explainer, shap_values = compute_xgb_shap_values(xgb_model, X_shap)

expected_value = explainer.expected_value
if isinstance(expected_value, (list, np.ndarray)):
    expected_value_scalar = float(np.array(expected_value).reshape(-1)
[0])
else:
    expected_value_scalar = float(expected_value)

mean_abs_shap = pd.Series(np.abs(shap_values).mean(axis=0),
index=X_shap.columns).sort_values(ascending=False)
mean_signed_shap = pd.Series(shap_values.mean(axis=0),
index=X_shap.columns).sort_values(key=np.abs, ascending=False)

display(mean_abs_shap.head(25).rename("mean_abs_shap").to_frame())
display(mean_signed_shap.head(25).rename("mean_signed_shap").to_frame(
))

plt.figure(figsize=(10, 8))
mean_abs_shap.head(20).sort_values().plot(kind="barh")
plt.title("Top 20 features by mean absolute SHAP")
plt.xlabel("Mean absolute SHAP")
savefig(cfg, "xgb_02_mean_abs_shap.png")
plt.show()

shap.summary_plot(shap_values, X_shap, max_display=20, show=False)
savefig(cfg, "xgb_03_shap_beeswarm.png")
plt.show()
```

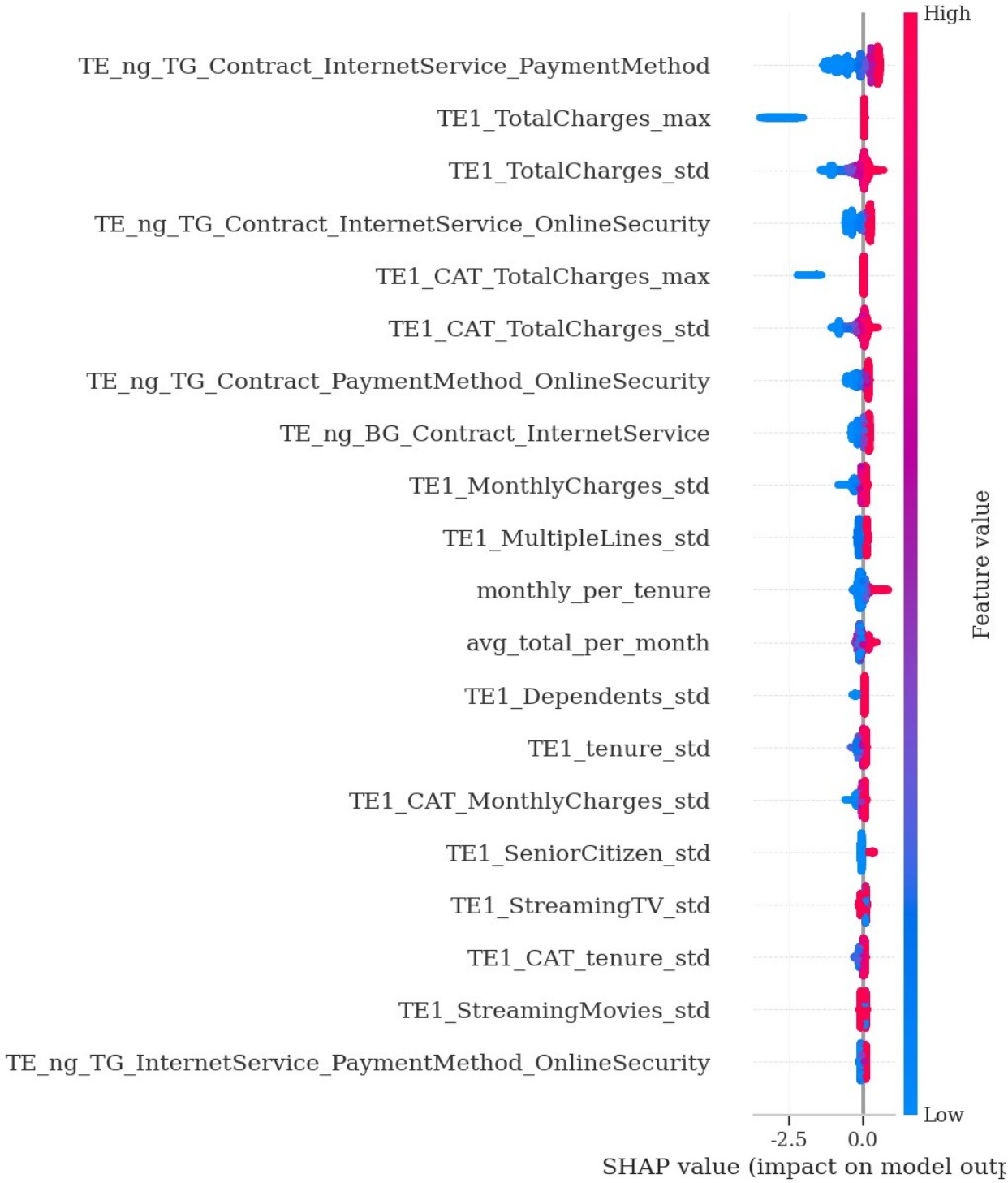
	mean_abs_shap
TE_ng_TG_Contract_InternetService_PaymentMethod	0.530066
TE1_TotalCharges_max	0.369112
TE1_TotalCharges_std	0.321982
TE_ng_TG_Contract_InternetService_OnlineSecurity	0.265316
TE1_CAT_TotalCharges_max	0.247238
TE1_CAT_TotalCharges_std	0.242335
TE_ng_TG_Contract_PaymentMethod_OnlineSecurity	0.233425
TE_ng_BG_Contract_InternetService	0.188321
TE1_MonthlyCharges_std	0.121252
TE1_MultipleLines_std	0.118988
monthly_per_tenure	0.117254
avg_total_per_month	0.115510
TE1_Dependents_std	0.104409
TE1_tenure_std	0.101957
TE1_CAT_MonthlyCharges_std	0.088678
TE1_SeniorCitizen_std	0.088612
TE1_StreamingTV_std	0.087497
TE1_CAT_tenure_std	0.075444
TE1_StreamingMovies_std	0.074691
TE_ng_TG_InternetService_PaymentMethod_OnlineSe...	0.070517
TE_ng_BG_TechSupport_PaperlessBilling	0.069025
TE_ng_BG_Contract_PaymentMethod	0.065014
TE_ng_BG_Contract_TechSupport	0.061391
TE_ng_BG_Contract_OnlineSecurity	0.060999
ORIG_proba_tenure	0.060460

	mean_signed_shap
TE1_TotalCharges_max	-0.311591
TE1_TotalCharges_std	-0.226661
TE_ng_TG_Contract_InternetService_PaymentMethod	-0.217541
TE1_CAT_TotalCharges_max	-0.209578
TE1_CAT_TotalCharges_std	-0.166938
TE_ng_TG_Contract_PaymentMethod_OnlineSecurity	-0.106774
TE_ng_TG_Contract_InternetService_OnlineSecurity	-0.103124
TE1_MonthlyCharges_std	-0.074320
TE1_tenure_std	-0.056286
TE1_CAT_MonthlyCharges_std	-0.055837
TE_ng_BG_Contract_InternetService	-0.051475
TE1_CAT_tenure_std	-0.041749
TE1_Dependents_std	-0.040867
TE_ng_BG_Contract_PaperlessBilling	-0.037763
TE_ng_BG_TechSupport_PaperlessBilling	-0.034329
TE_ng_BG_Contract_PaymentMethod	-0.031341
monthly_per_tenure	-0.029235
TE_ng_TG_InternetService_PaymentMethod_OnlineSe...	-0.029133
mc_per_digit	0.027419
avg_total_per_month	-0.025832
ORIG_proba_tenure	-0.025157
TE_ng_BG_PaymentMethod_PaperlessBilling	-0.024866

```
TE1_SeniorCitizen_std                                          -0.023626
TE_ng_BG_Contract_TechSupport                                  -0.023061
TE_ng_BG_Contract_OnlineSecurity                               -0.022455

saved: analysis_outputs\xgb_02_mean_abs_shap.png
```




```
saved: analysis_outputs\xgb_03_shap_beeswarm.png
```



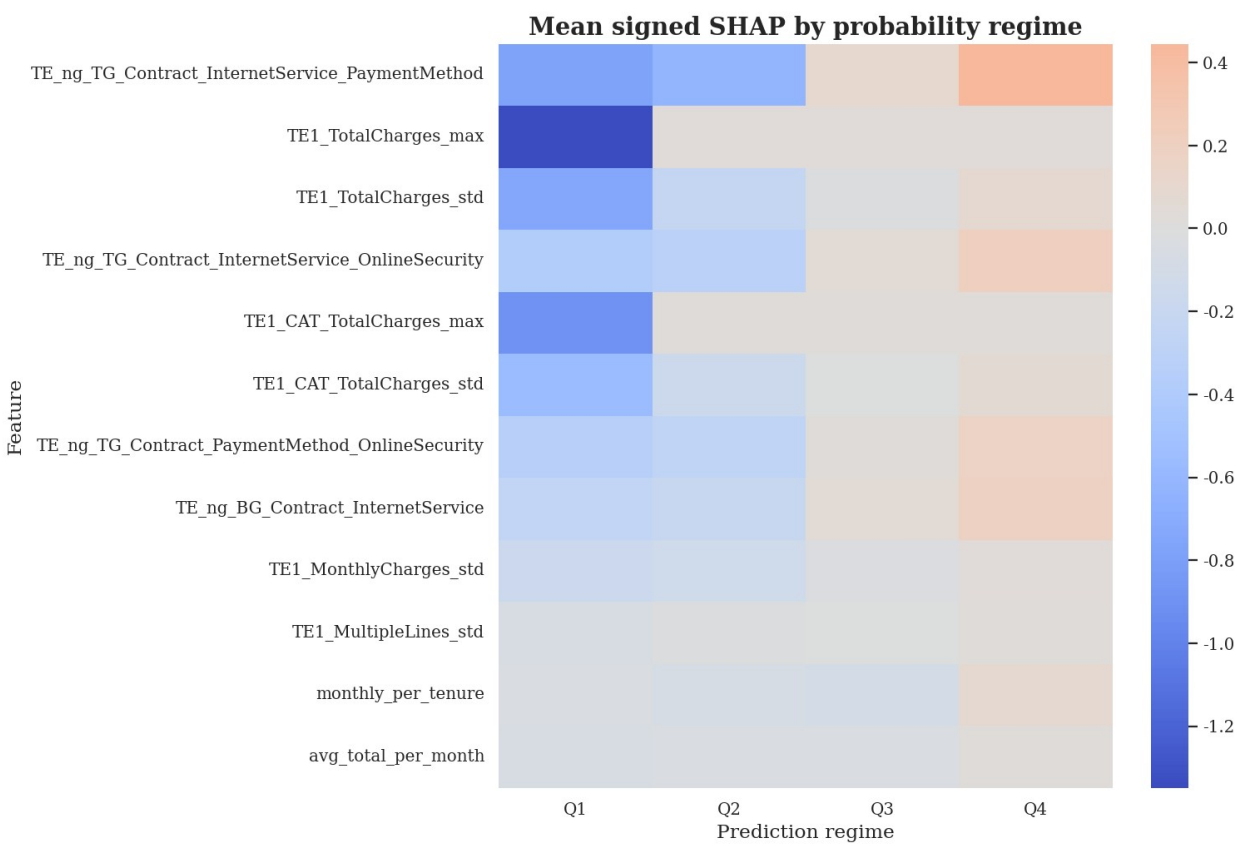
Mathematical framing

In this section, I study the model as a function $\hat{p}(x)$ mapping inputs to probabilities. I interpret each computation as evaluating properties of this function such as gradients, perturbations, or expectations.

4. Signed SHAP by prediction regime

A feature may be globally important but behave differently in low-risk and high-risk regimes. We therefore compare the mean signed SHAP contributions inside probability bands.

```
shap_df = pd.DataFrame(shap_values, columns=X_shap.columns)
regime = pd.qcut(xgb_model.predict_proba(X_shap)[:, 1], q=4,
labels=["Q1", "Q2", "Q3", "Q4"], duplicates="drop")
shap_df["regime"] = regime.to_numpy()

top20 = mean_abs_shap.head(12).index.tolist()
regime_signed = shap_df.groupby("regime")[top20].mean().T

plt.figure(figsize=(12, 8))
sns.heatmap(regime_signed, cmap="coolwarm", center=0)
plt.title("Mean signed SHAP by probability regime")
plt.xlabel("Prediction regime")
plt.ylabel("Feature")
savefig(cfg, "xgb_04_regime_signed_shap_heatmap.png")
plt.show()

display(regime_signed)

saved: analysis_outputs\xgb_04_regime_signed_shap_heatmap.png
```



```
regime                                                    Q1
Q2  \
TE_ng_TG_Contract_InternetService_PaymentMethod  -0.780428 -0.629880

TE1_TotalCharges_max                             -1.348533  0.032095

TE1_TotalCharges_std                             -0.744716 -0.225837

TE_ng_TG_Contract_InternetService_OnlineSecurity -0.373592 -0.312692

TE1_CAT_TotalCharges_max                         -0.904992  0.021995

TE1_CAT_TotalCharges_std                         -0.559022 -0.163217

TE_ng_TG_Contract_PaymentMethod_OnlineSecurity   -0.344426 -0.272357

TE_ng_BG_Contract_InternetService                -0.242659 -0.202735

TE1_MonthlyCharges_std                           -0.175661 -0.132663

TE1_MultipleLines_std                            -0.055214 -0.020275

monthly_per_tenure                               -0.042444 -0.067339

avg_total_per_month                              -0.058950 -0.039603


regime                                                   Q3        Q4
TE_ng_TG_Contract_InternetService_PaymentMethod   0.095037  0.445107
TE1_TotalCharges_max                              0.033655  0.036423
TE1_TotalCharges_std                             -0.016901  0.080809
TE_ng_TG_Contract_InternetService_OnlineSecurity  0.058432  0.215357
TE1_CAT_TotalCharges_max                          0.021818  0.022867
TE1_CAT_TotalCharges_std                         -0.008501  0.062989
TE_ng_TG_Contract_PaymentMethod_OnlineSecurity    0.023543  0.166144
TE_ng_BG_Contract_InternetService                 0.056969  0.182526
TE1_MonthlyCharges_std                           -0.026953  0.037998
TE1_MultipleLines_std                            -0.007462  0.018923
monthly_per_tenure                               -0.100796  0.093637
avg_total_per_month                              -0.032367  0.027592
```

Mathematical framing

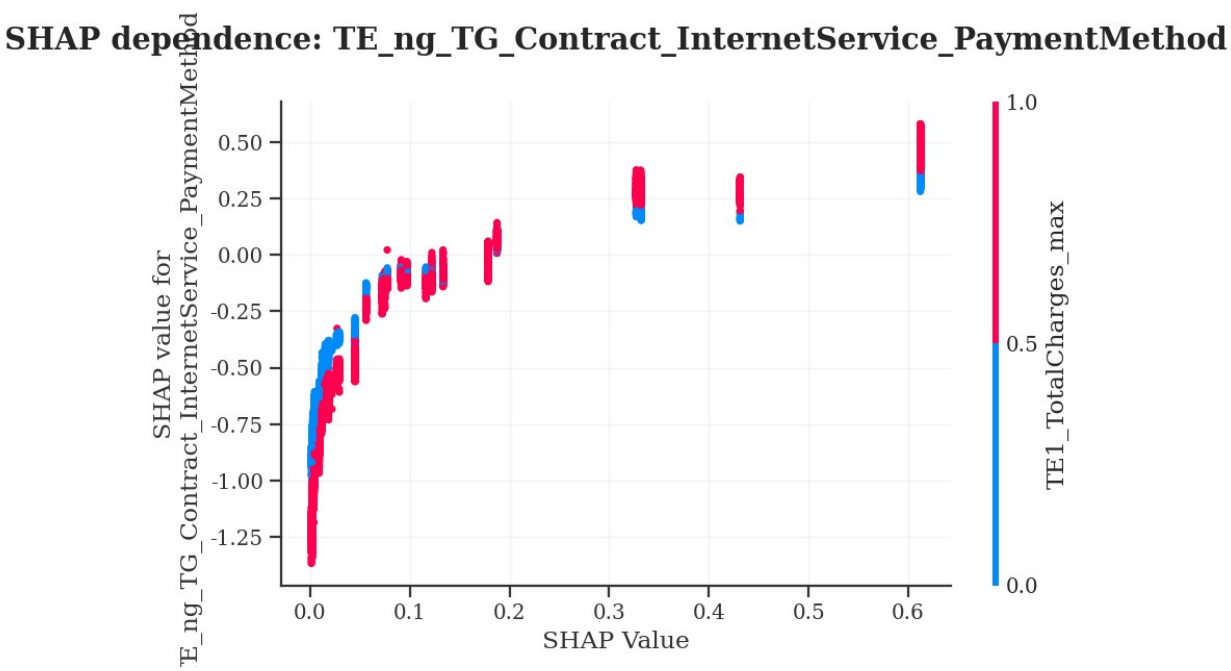
In this section, I study the model as a function $\hat{p}(x)$ mapping inputs to probabilities. I interpret each computation as evaluating properties of this function such as gradients, perturbations, or expectations.

5. Dependence structure and interaction candidates

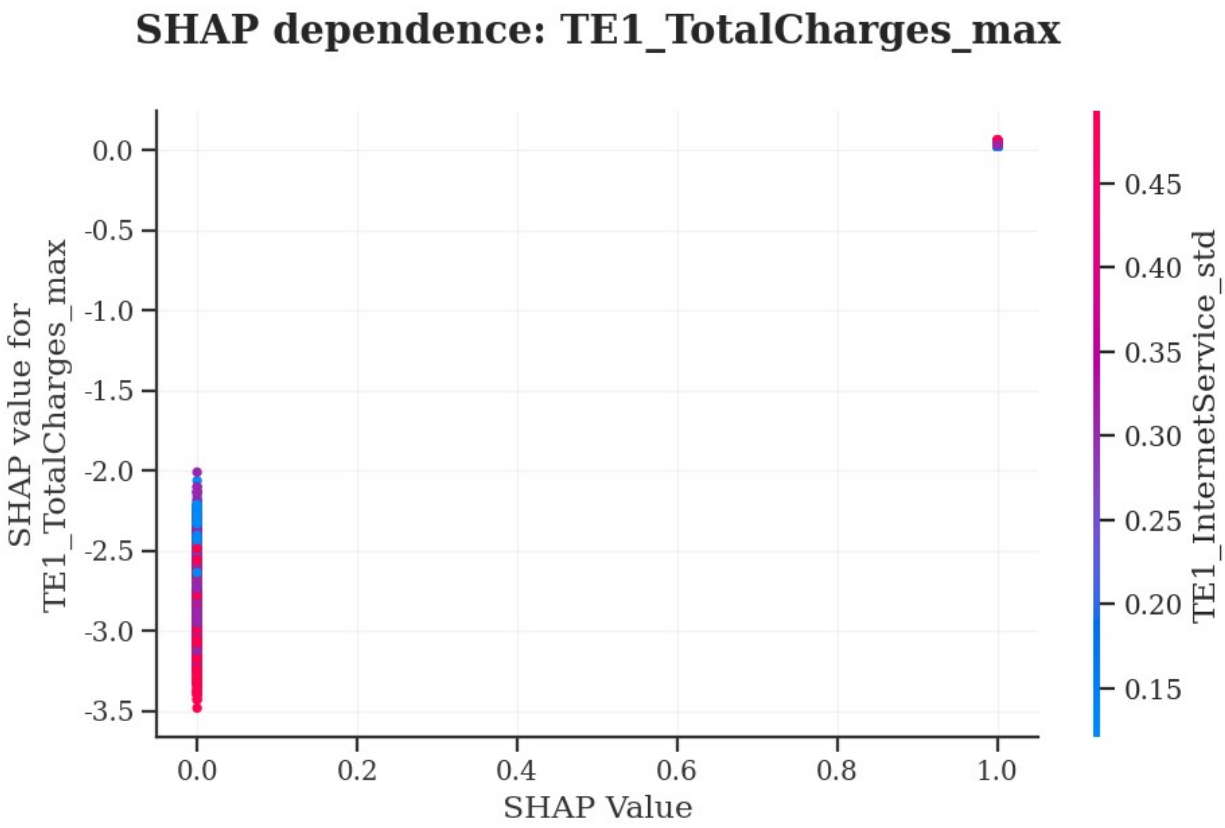
For top SHAP features, we inspect dependence plots. For trees, visible vertical jumps or terracing are expected because the underlying function is split-based and piecewise constant on the logit scale.

```
top_features = mean_abs_shap.head(8).index.tolist()

for feat in top_features[:6]:
    shap.dependence_plot(
        feat,
        shap_values,
        X_shap,
        interaction_index="auto",
        show=False,
    )
    # Move the title a bit higher up and remove x axis label
    plt.title(f"SHAP dependence: {feat}", pad=28)
    plt.xlabel("SHAP Value")
    fname = f"xgb_dep_{feat.replace('/', '_').replace(' ', '_')}.png"
    savefig(cfg, fname)
    plt.show()
```

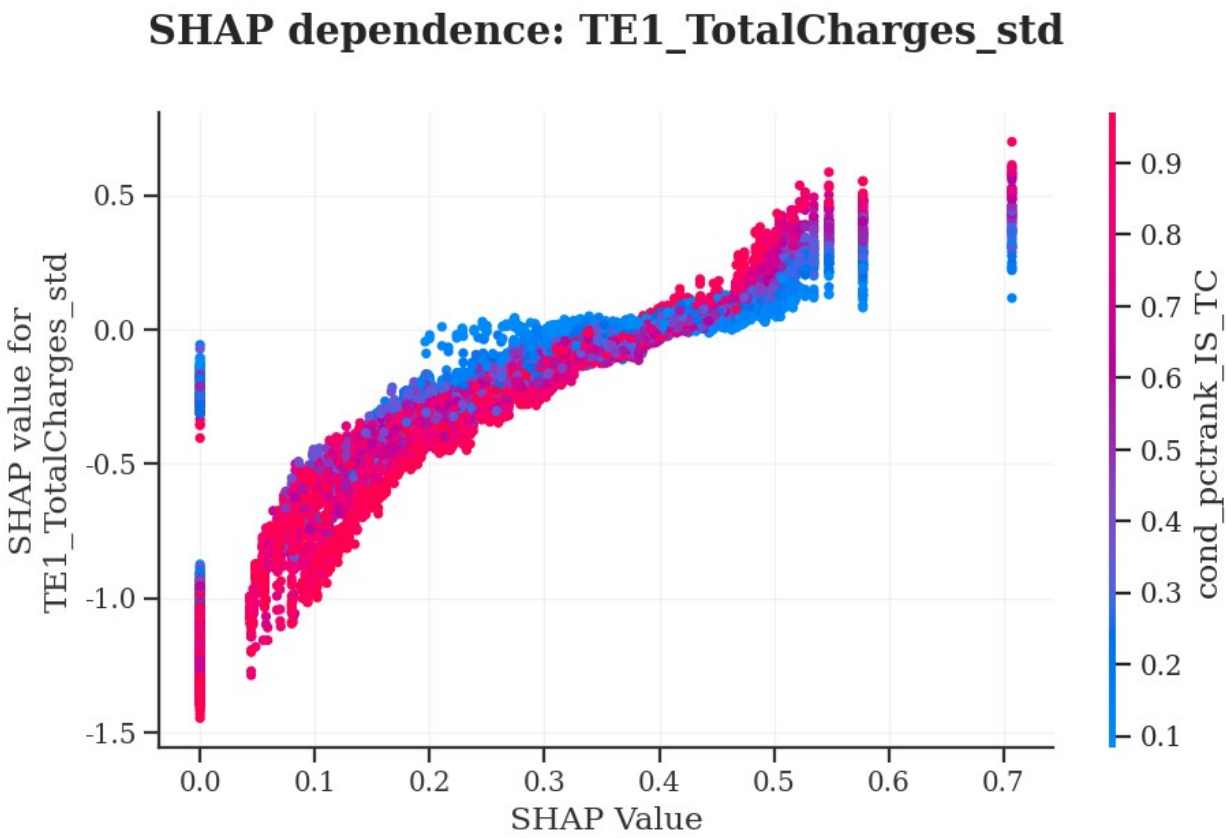
```
saved: analysis_outputs\
xgb_dep_TE_ng_TG_Contract_InternetService_PaymentMethod.png
```



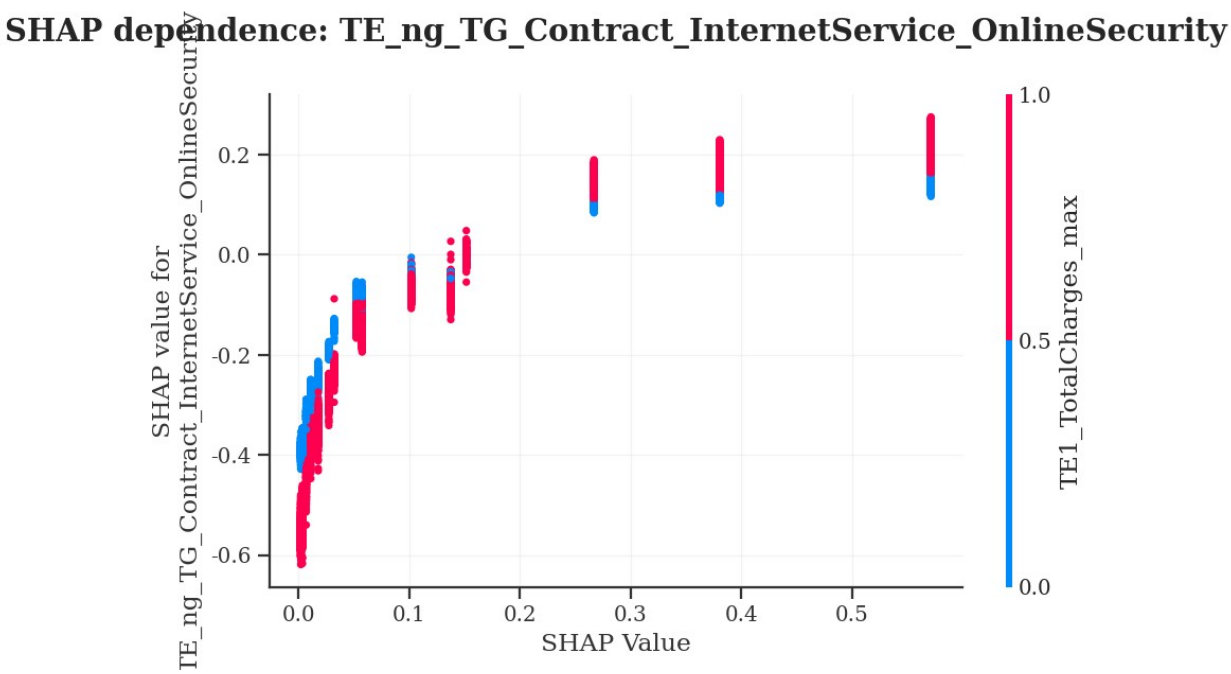

```
saved: analysis_outputs\xgb_dep_TE1_TotalCharges_max.png
```



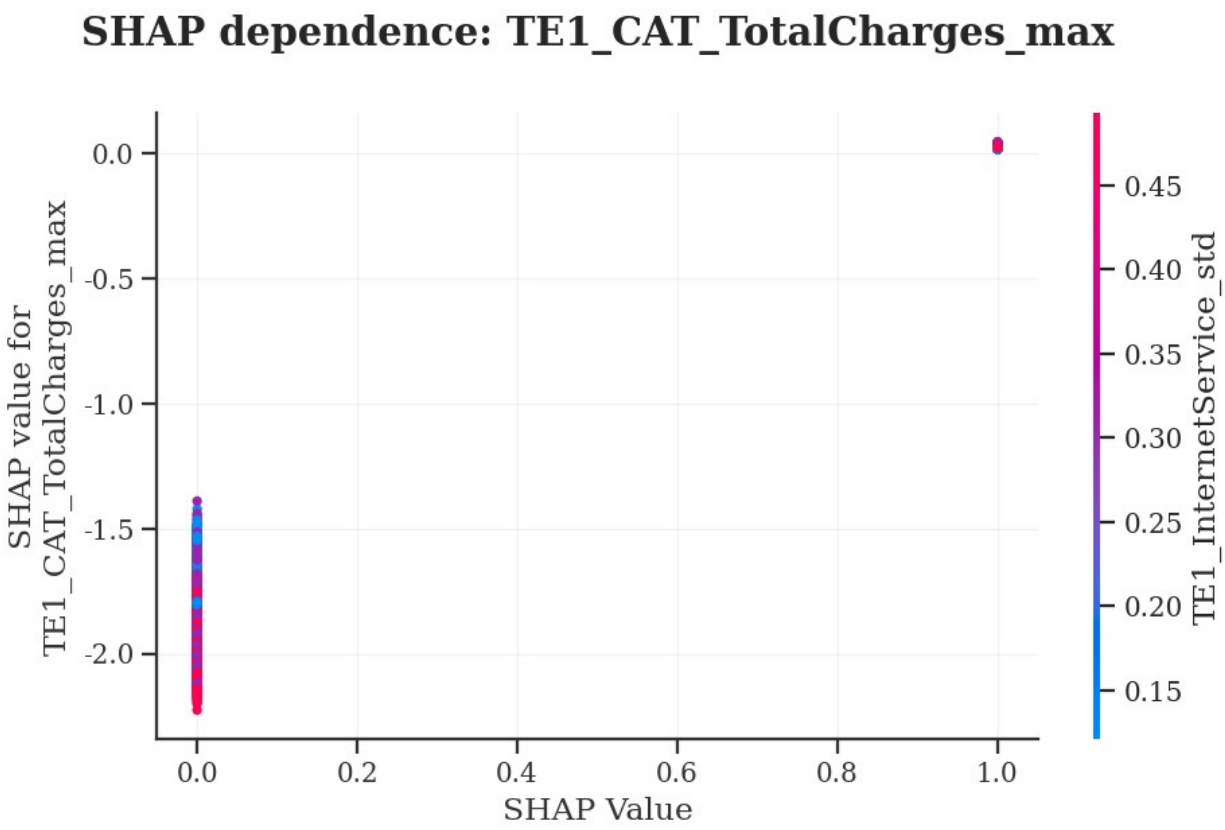

```
saved: analysis_outputs\xgb_dep_TE1_TotalCharges_std.png
```



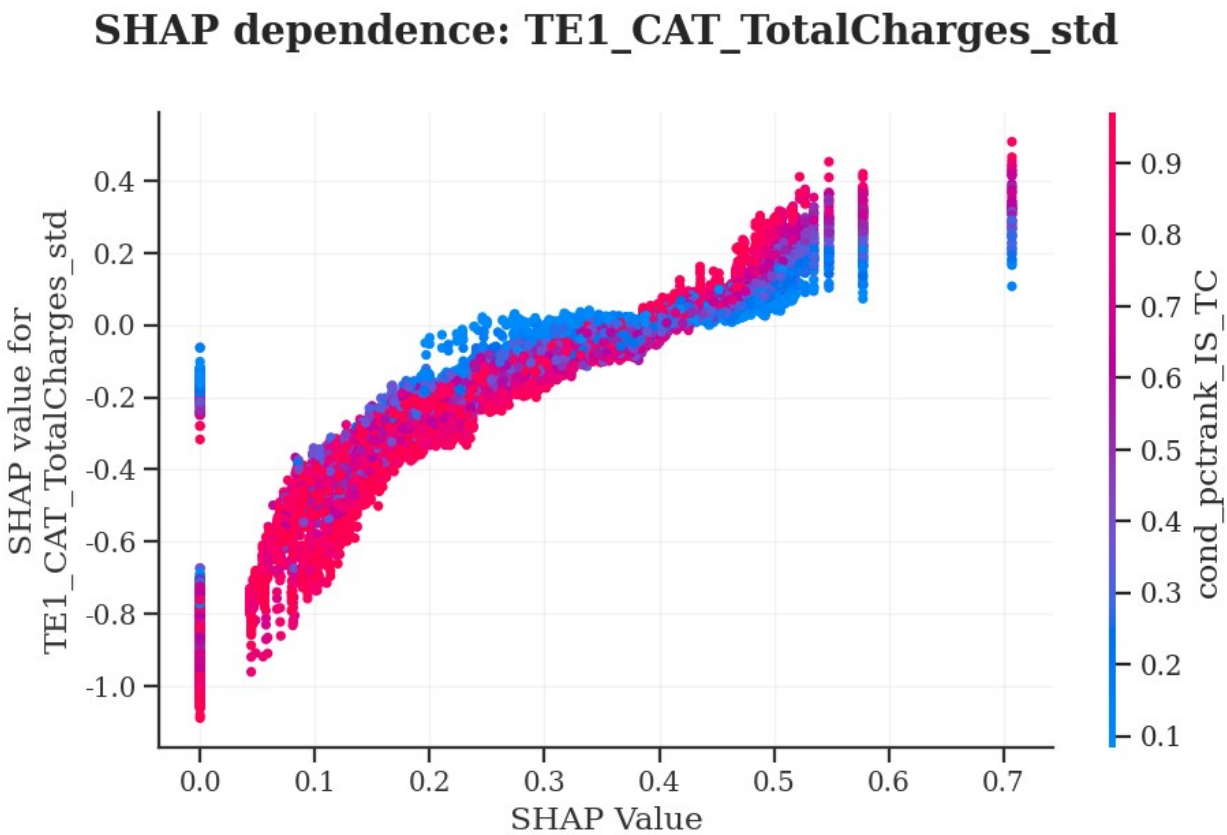

```
saved: analysis_outputs\
xgb_dep_TE_ng_TG_Contract_InternetService_OnlineSecurity.png
```



```
saved: analysis_outputs\xgb_dep_TE1_CAT_TotalCharges_max.png
```




```
saved: analysis_outputs\xgb_dep_TE1_CAT_TotalCharges_std.png
```




```
interaction_candidates = []
for feat in top_features[:6]:
    try:
        partners = shap.utils.approximate_interactions(feat,
shap_values, X_shap)
        if len(partners) > 1:
            partner = X_shap.columns[partners[1]]
        else:
            partner = X_shap.columns[partners[0]]
        interaction_candidates.append((feat, partner))
    except Exception:
        pass

interaction_candidates = pd.DataFrame(interaction_candidates,
columns=["feature", "approx_partner"]).drop_duplicates()
display(interaction_candidates)
```

```
                                                feature  \
0   TE_ng_TG_Contract_InternetService_PaymentMethod
1                              TE1_TotalCharges_max
2                              TE1_TotalCharges_std
3  TE_ng_TG_Contract_InternetService_OnlineSecurity
4                          TE1_CAT_TotalCharges_max
5                          TE1_CAT_TotalCharges_std
```

```
                 approx_partner
0    TE1_CAT_TotalCharges_max
1  ORIG_proba_InternetService
2           ORIG_proba_tenure
3    TE1_CAT_TotalCharges_max
4  ORIG_proba_InternetService
5           ORIG_proba_tenure
```

Mathematical framing

In this section, I study the model as a function $\hat{p}(x)$ mapping inputs to probabilities. I interpret each computation as evaluating properties of this function such as gradients, perturbations, or expectations.

6. Conditional decision surfaces

A tabular decision boundary cannot be visualised directly in full dimension. We therefore inspect two-dimensional conditional slices:

$$(x_i, x_j) \mapsto p_{\text{xgb}}(x_i, x_j, \tilde{x}_{-ij}),$$

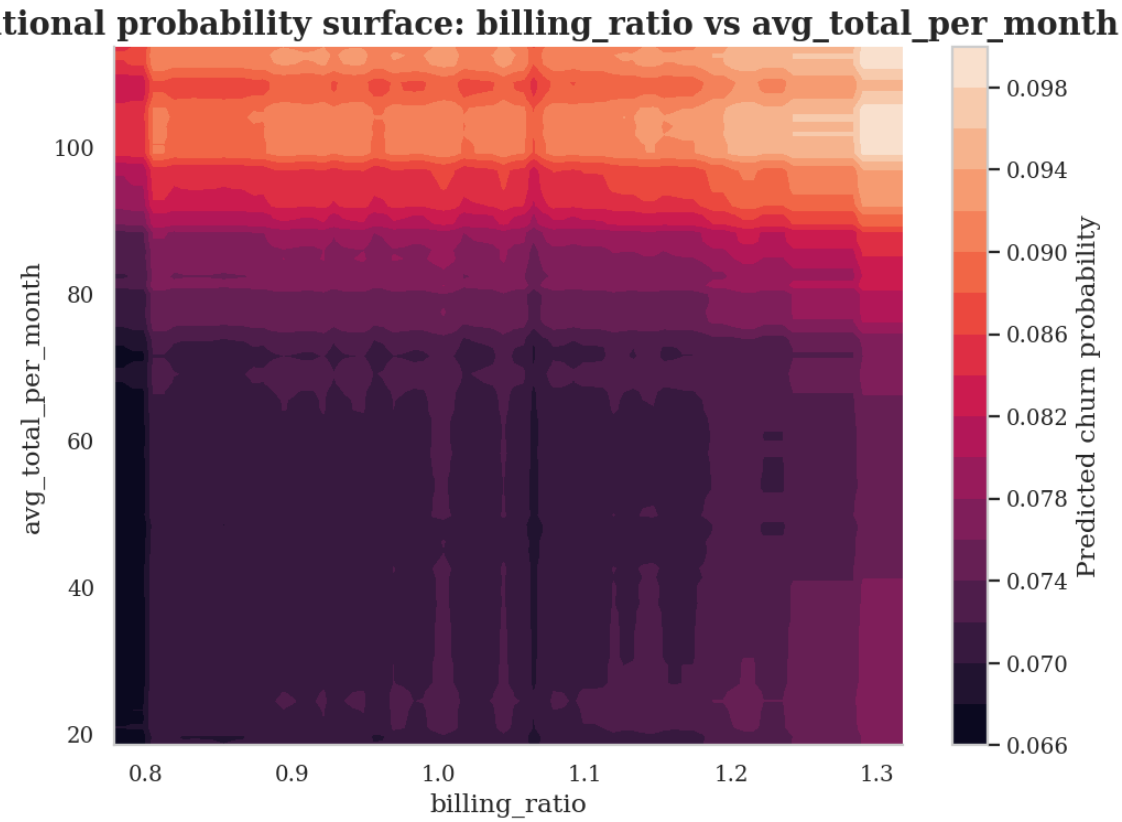
where all non-focal coordinates are fixed at a reference profile.

The resulting surfaces should reveal staircase-like geometry if the tree ensemble is using threshold partitions sharply.

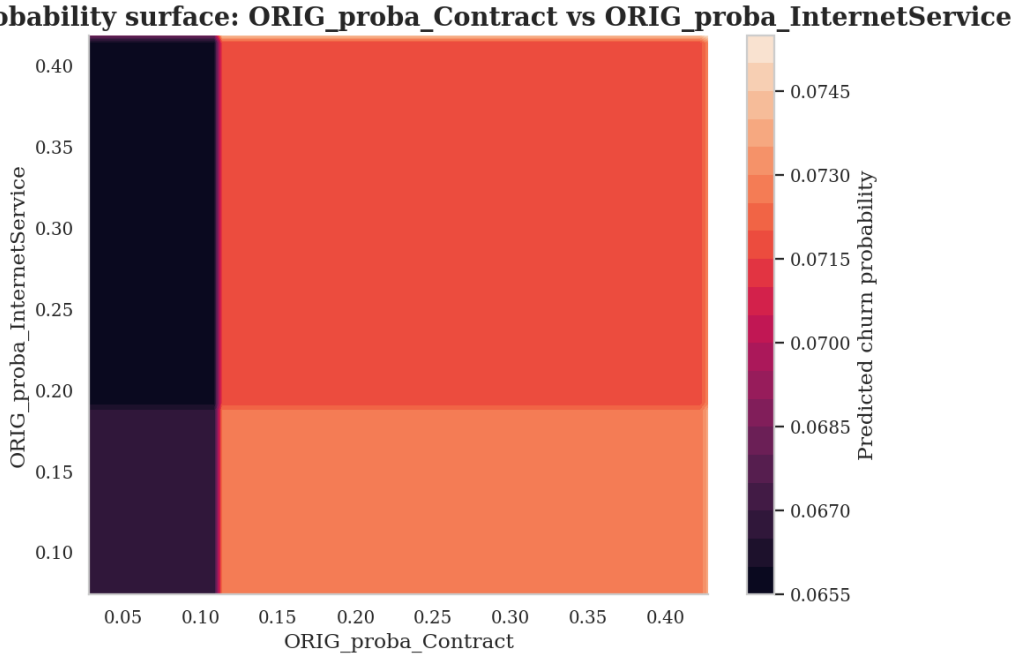
```
def make_reference_row(X):
    ref = {}
    for c in X.columns:
        s = X[c]
        if pd.api.types.is_numeric_dtype(s):
            ref[c] = float(s.median())
        else:
            mode = s.mode(dropna=True)
            ref[c] = mode.iloc[0] if len(mode) else s.iloc[0]
    return pd.Series(ref)

def plot_xgb_surface(model, X_ref, feature_x, feature_y, q_low=0.02,
q_high=0.98, grid_n=80):
    row = make_reference_row(X_ref)
    xs = np.linspace(X_ref[feature_x].quantile(q_low),
X_ref[feature_x].quantile(q_high), grid_n)
    ys = np.linspace(X_ref[feature_y].quantile(q_low),
X_ref[feature_y].quantile(q_high), grid_n)

    mesh = []
    for yv in ys:
        tmp = pd.DataFrame([row] * len(xs))
```

```
        tmp[feature_x] = xs
        tmp[feature_y] = yv
        mesh.append(tmp)
    mesh = pd.concat(mesh, axis=0, ignore_index=True)
    z = model.predict_proba(mesh[X_ref.columns])[:,
1].reshape(len(ys), len(xs))

    fig, ax = plt.subplots(figsize=(8, 6))
    cf = ax.contourf(xs, ys, z, levels=20)
    ax.contour(xs, ys, z, levels=[0.5], linewidths=2)
    ax.set_xlabel(feature_x)
    ax.set_ylabel(feature_y)
    ax.set_title(f"Conditional probability surface: {feature_x} vs
{feature_y}")
    plt.colorbar(cf, ax=ax, label="Predicted churn probability")
    return fig, ax

surface_pairs = []
for a, b in [
    ("tenure", "MonthlyCharges"),
    ("billing_ratio", "avg_total_per_month"),
    ("ORIG_proba_Contract", "ORIG_proba_InternetService"),
    ("tenure_mod12", "MonthlyCharges"),
]:
    if a in X_train_aligned.columns and b in X_train_aligned.columns:
        surface_pairs.append((a, b))

for a, b in surface_pairs:
    fig, ax = plot_xgb_surface(xgb_model,
X_train_aligned.iloc[:n_comp], a, b)
    savefig(cfg, f"xgb_surface_{a}_{b}.png".replace("/",
"_").replace(" ", "_"))
    plt.show()
```

```
saved: analysis_outputs\
xgb_surface_billing_ratio_avg_total_per_month.png
```




```
saved: analysis_outputs\
xgb_surface_ORIG_proba_Contract_ORIG_proba_InternetService.png
```



Mathematical framing

In this section, I study the model as a function $\hat{p}(x)$ mapping inputs to probabilities. I interpret each computation as evaluating properties of this function such as gradients, perturbations, or expectations.

8. Error-specific explanation

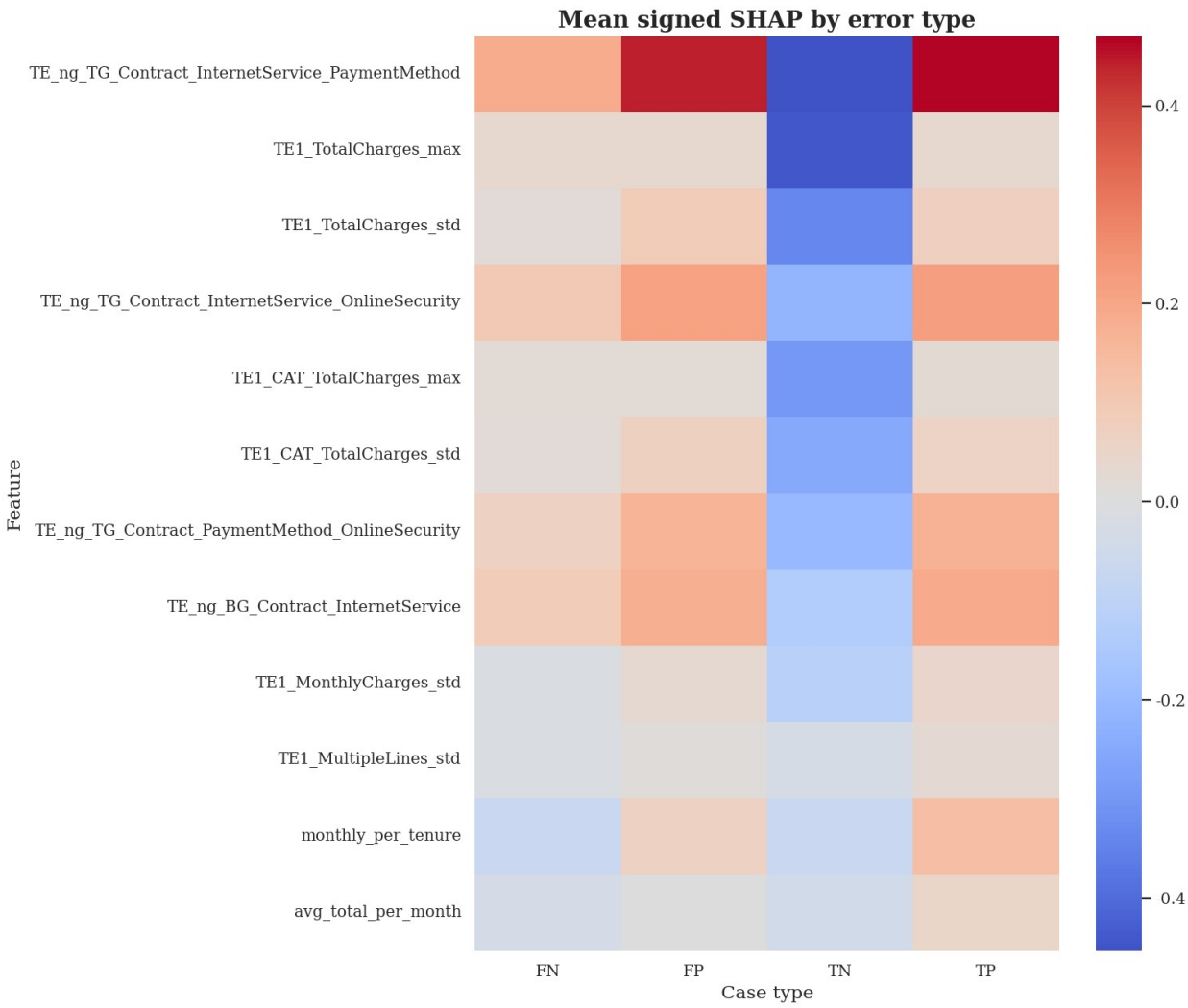
We now compare SHAP structures for:

- false positives,
- false negatives,
- true positives,
- true negatives.

This isolates where the model's inductive bias is helping and where it is misallocating risk.

```
shap_analysis = analysis_train.loc[shap_idx, [cfg.id_col, "y_true",
"xgb_pred",
"case_type"]].reset_index(drop=True).join(shap_df.drop(columns="regime
"))
group_means = shap_analysis.groupby("case_type")[top20].mean().T

plt.figure(figsize=(12, 10))
sns.heatmap(group_means, cmap="coolwarm", center=0)
plt.title("Mean signed SHAP by error type")
plt.xlabel("Case type")
plt.ylabel("Feature")
savefig(cfg, "xgb_05_signed_shap_by_case_type.png")
plt.show()

display(group_means)
```

```
saved: analysis_outputs\xgb_05_signed_shap_by_case_type.png
```




```
case_type                                                    FN
FP  \
TE_ng_TG_Contract_InternetService_PaymentMethod   0.187166  0.441763

TE1_TotalCharges_max                              0.034215  0.035932

TE1_TotalCharges_std                              0.016664  0.090646

TE_ng_TG_Contract_InternetService_OnlineSecurity  0.102045  0.212919

TE1_CAT_TotalCharges_max                          0.021921  0.022577

TE1_CAT_TotalCharges_std                          0.016465  0.071195

TE_ng_TG_Contract_PaymentMethod_OnlineSecurity    0.065052  0.165380

TE_ng_BG_Contract_InternetService                 0.088950  0.180518
```

```
TE1_MonthlyCharges_std                                -0.011059  0.032678

TE1_MultipleLines_std                                 -0.009929  0.011905

monthly_per_tenure                                    -0.069810  0.067704

avg_total_per_month                                   -0.032325  0.003988


case_type                                                    TN        TP
TE_ng_TG_Contract_InternetService_PaymentMethod  -0.453075  0.470365
TE1_TotalCharges_max                              -0.445589  0.036968
TE1_TotalCharges_std                              -0.338856  0.077007
TE_ng_TG_Contract_InternetService_OnlineSecurity -0.216704  0.224721
TE1_CAT_TotalCharges_max                          -0.299120  0.023171
TE1_CAT_TotalCharges_std                          -0.250898  0.059479
TE_ng_TG_Contract_PaymentMethod_OnlineSecurity   -0.203879  0.174460
TE_ng_BG_Contract_InternetService                 -0.134375  0.191582
TE1_MonthlyCharges_std                            -0.114037  0.045382
TE1_MultipleLines_std                             -0.027966  0.027394
monthly_per_tenure                                -0.067831  0.135724
avg_total_per_month                               -0.043842  0.050836
```

Mathematical framing

In this section, I study the model as a function $\hat{p}(x)$ mapping inputs to probabilities. I interpret each computation as evaluating properties of this function such as gradients, perturbations, or expectations.

9. Local explanations

For representative cases, we inspect full SHAP decompositions.

The local decomposition is

$$f_{\text{xgb}}(x^{(i)}) = \phi_0 + \sum_{j=1}^{p} \phi_j(x^{(i)}).$$

We deliberately choose extreme true positives, extreme true negatives, a false positive, and a false negative where available.

```
candidate_indices = []

tp_idx = analysis_train.query("case_type == 'TP'").sort_values("xgb_pred", ascending=False).index
tn_idx = analysis_train.query("case_type == 'TN'").sort_values("xgb_pred", ascending=True).index
fp_idx = analysis_train.query("case_type == 'FP'").sort_values("xgb_pred", ascending=False).index
```

```
fn_idx = analysis_train.query("case_type ==
'FN'").sort_values("xgb_pred", ascending=True).index

for arr in [tp_idx, tn_idx, fp_idx, fn_idx]:
    if len(arr):
        candidate_indices.append(int(arr[0]))

candidate_indices = [i for i in candidate_indices if i in
set(shap_idx.tolist())]
candidate_positions = [int(np.where(shap_idx == i)[0][0]) for i in
candidate_indices]

for pos in candidate_positions:
    row_meta = analysis_train.loc[shap_idx[pos], ["y_true",
"xgb_pred", "case_type"]]
    expl = shap.Explanation(
        values=shap_values[pos],
        base_values=expected_value_scalar,
        data=X_shap.iloc[pos].values,
        feature_names=X_shap.columns.tolist(),
    )
    print(row_meta.to_dict())
    shap.plots.waterfall(expl, max_display=20, show=False)
    savefig(cfg, f"xgb_waterfall_row_{int(shap_idx[pos])}.png")
    plt.show()
```

Mathematical framing

In this section, I study the model as a function $\hat{p}(x)$ mapping inputs to probabilities. I interpret each computation as evaluating properties of this function such as gradients, perturbations, or expectations.

10. Export tables

The notebook writes the central outputs needed for a report:

- fitted train/test predictions,
- decile table,
- SHAP rankings,
- regime-specific SHAP summaries,
- case-type SHAP summaries.

```
outdir = Path(cfg.output_dir)

analysis_train.to_csv(outdir / "xgb_analysis_train.csv", index=False)
analysis_test.to_csv(outdir / "xgb_analysis_test.csv", index=False)
dec.to_csv(outdir / "xgb_deciles.csv", index=False)
mean_abs_shap.rename("mean_abs_shap").to_csv(outdir /
"xgb_mean_abs_shap.csv")
```

```
mean_signed_shap.rename("mean_signed_shap").to_csv(outdir /
"xgb_mean_signed_shap.csv")
regime_signed.to_csv(outdir / "xgb_regime_signed_shap.csv")
group_means.to_csv(outdir / "xgb_case_type_signed_shap.csv")
interaction_candidates.to_csv(outdir /
"xgb_interaction_candidates.csv", index=False)

summary = {
    "train_auc_competition_only": float(roc_auc_score(y,
xgb_comp_train_pred)),
    "train_logloss_competition_only": float(log_loss(y,
xgb_comp_train_pred)),
    "train_brier_competition_only": float(brier_score_loss(y,
xgb_comp_train_pred)),
    "n_saved_features": int(len(feature_cols_saved)),
    "n_runtime_rows_train": int(X_train_aligned.shape[0]),
    "n_runtime_rows_test": int(X_test_aligned.shape[0]),
    "n_runtime_cols": int(X_train_aligned.shape[1]),
    "top10_features_mean_abs_shap": mean_abs_shap.head(10).to_dict(),
}

(outdir / "xgb_xai_summary.json").write_text(json.dumps(summary,
indent=2))
summary
```

```
{'train_auc_competition_only': 0.9334367672728091,
 'train_logloss_competition_only': 0.26805529743260703,
 'train_brier_competition_only': 0.0859934464097023,
 'n_saved_features': 161,
 'n_runtime_rows_train': 601237,
 'n_runtime_rows_test': 254655,
 'n_runtime_cols': 161,
 'top10_features_mean_abs_shap':
{'TE_ng_TG_Contract_InternetService_PaymentMethod':
0.5300655364990234,
  'TE1_TotalCharges_max': 0.3691123425960541,
  'TE1_TotalCharges_std': 0.3219815790653229,
  'TE_ng_TG_Contract_InternetService_OnlineSecurity':
0.2653157413005829,
  'TE1_CAT_TotalCharges_max': 0.2472381889820099,
  'TE1_CAT_TotalCharges_std': 0.24233508110046387,
  'TE_ng_TG_Contract_PaymentMethod_OnlineSecurity':
0.23342451453208923,
  'TE_ng_BG_Contract_InternetService': 0.18832050263881683,
  'TE1_MonthlyCharges_std': 0.12125235795974731,
  'TE1_MultipleLines_std': 0.11898791790008545}}
```